01 ASSIGNMENT 2 — PRACTICAL 1

ASSIGNMENT 2 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, MaxPooling2D, Flatten
from tensorflow.keras.optimizers import SGD

# LOAD DATASET
data = pd.read_csv("cifar-10.csv")

# TRAIN TEST SPLIT
X = data.drop("label", axis = 1).values
y = data["label"].values

# RESHAPE IMAGES INTO RGB FORMAT
X = X.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)

# NORMALIZE
X = X.astype("float32") / 255.0

# TRAIN TEST SPLITTING
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state = 42 )

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

# DEFINING THE ARCHITECTURE
model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(32,32,3)),
    MaxPooling2D((2,2)),

    Conv2D(32, (3,3), activation="relu" ),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)

# Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# Accuracy graph
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss graph
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Predict the output
predictions = model.predict(X_test)

y_pred = np.argmax(predictions, axis = 1 )
y_true = y_test

print("Predicted Digit: ", y_pred[:10])
print("Acutal Digit: ", y_true[:10])
```

02 ASSIGNMENT 2 — PRACTICAL 2

ASSIGNMENT 2 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import models
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam

# LOAD DATASETS
train_data = pd.read_csv("mnist_train.csv")
test_data = pd.read_csv("mnist_test.csv")

# SEPARATE FEATURES AND LABELS
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# RESHAPE IMAGES INTO 28*28 grayscale format
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# NORMALIZE PIXEL VALUES
X_train = X_train.astype("float32")/255.0
X_test = X_test.astype("float32")/255.0

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

# DEFINING ARCHITECTURE OF MODEL
model = models.Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(32, (3,3), activation="relu"),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer = Adam(learning_rate=0.001),
    loss = "sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=11,
    batch_size=32,
    validation_split = 0.2
)

# ESTIMATING MODEL PERFORMANCE
```

```
test_loss , test_accuracy = model.evaluate(X_test, y_test)

print("Test loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# Accuracy graph
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss graph
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Predict the output
predictions = model.predict(X_test)

y_pred = np.argmax(predictions, axis=1)

print("Predicted Digit:", y_pred[0])
print("Actual Digit:", y_test[0])

plt.imshow(X_test[0].reshape(28,28), cmap='gray')
plt.title("Predicted: " + str(y_pred[0]) +
          " | Actual: " + str(y_test[0]))
plt.axis('off')
plt.show()
```

03 ASSIGNMENT 2 — PRACTICAL 3

ASSIGNMENT 2 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

from tensorflow.keras import models
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
from tensorflow.keras.optimizers import Adam

# LOADING DATASET
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")

# SEPARATE FEATURE AND LABELS
X_train = train_data.drop("label",axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

encoder = LabelEncoder()

y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)

# RESHAPE IMAGES INTO 28*28 GRAYSCALE
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Normalize pixel values
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
print("Number of classes:", len(encoder.classes_))

# DEFINING THE ARCHITECTURE
model = models.Sequential([
    # First convolution layer
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    # Second convolution layer
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    # Classification layers
    Flatten(),
    Dense(64, activation='relu'),
    Dense(24, activation='softmax')
])

model.summary()

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
```

```
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)

# Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# Accuracy graph
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss graph
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Predict the output
predictions = model.predict(X_test)

y_pred = np.argmax(predictions, axis=1)

print("Predicted Label:", encoder.inverse_transform([y_pred[0]])[0])
print("Actual Label:", encoder.inverse_transform([y_test[0]])[0])

plt.imshow(X_test[0].reshape(28,28), cmap='gray')

predicted = encoder.inverse_transform([y_pred[0]])[0]
actual = encoder.inverse_transform([y_test[0]])[0]

plt.title("Predicted: " + str(predicted) +
          " | Actual: " + str(actual))

plt.axis('off')
plt.show()
```